

Examen réparti 1

Master 1 SAR

UE MU4IN401

L. Arantes, P. Sens, J. Sopena

Novembre 2023

2 heures – **Documents non autorisés**

Tout appareil électronique interdit

Le barème est donné à titre indicatif

Exercice 1 – Questions de cours (3 points)

Question 1 $\frac{1}{2}$ point

Lors d'un appel système, un processus s'exécute-t-il sur sa pile utilisateur ?

Solution:

non sur la pile système.

Question 2 $\frac{1}{2}$ point

Est-il possible qu'un même code soit présent en deux exemplaires en mémoire ?

Solution:

non si plusieurs processus utilisent le même code, il y a une seule copie.

Question 3 $\frac{1}{2}$ point

Quelles fonctions du noyau positionnent la variable runrun à une valeur différente de 0 ?

Solution:

realtime-setpri et setrun

Question 4 $1\frac{1}{2}$ points

On considère un processus P1 de priorité PUSER=100 à l'état SRUN et un processus P2 de priorité 80 à l'état SSLEEP.

P1 est élu à $t=20$ secondes, on suppose qu'après 0,5 seconde il fait un appel système qui doit s'exécuter pendant 0,3 seconde. A $t=20,6$ secondes une interruption disque réveille P2. Le traitement de l'interruption disque dure 0,1 seconde. A quel moment P2 sera élu au plus tôt. Dessinez un chronogramme pour justifier votre réponse.

Solution:

P2 élu à $t=20,6+0,1+0,2 = 20,9$ s (date IT + durée traitement IT + fin traitement appel système)

Exercice 2 – Callout (3,5 points)

On considère l'Unix du TD avec la table globale callout de type `struct *callo` qui gère l'ensemble des callout.

```
struct callo
{
    int      c_time;           /* incremental time */
    caddr_t  c_arg;           /* argument to routine */
    int      (*c_func)();     /* routine */
};
```

Question 1 1 point

Initialement, la table ne contient aucune fonction. Donnez l'état de la table suite aux 4 appels fait dans cet ordre :

```
timeout(func1, 0, 10);
timeout(func2, 0, 10);
timeout(func3, 0, 20);
timeout(func4, 0, 25);
```

Solution:

func1	0	10
func2	0	0
func3	0	10
func4	0	5
0	-	-

Question 2 1 point

On fait l'appel suivant 2 ticks après les 4 premiers appels à timeout :

```
timeout(func5, 0, 11);
```

Donnez l'état de la table après cet appel.

Solution:

func1	0	8
func2	0	0
func5	0	3
func3	0	7
func4	0	5
0	-	-

Question 3 1 point

En supposant que le traitement de l'interruption horloge soit terminé, donnez l'état de la table 9 ticks après l'appel à timeout de la Question 2.

Solution:

func5	0	2
func3	0	7
func4	0	5
0	-	-

Question 4 1/2 point

Les fonctions func1, func2, func3, func4 et func5 s'exécutent elles en mode utilisateur ou système ?

Solution:

Système

Exercice 3 – Signaux et appel système (5,5 points)

Les codes des fonctions traitant les signaux sont rappelés en annexe. On considère un processus P ayant 4 signaux pendants (non encore traités), les signaux 2, 5, 6 et 7.

Question 1 $\frac{1}{2}$ point

Représentez en binaire la configuration du champs p_sig de la structure proc de P.

Solution:

15	...	7	6	5	4	3	2	1	0
0	...	0	1	1	1	0	0	1	0

Question 2 1 point

On considère que les signaux 5 et 7 sont ignorés. Les handlers des signaux 6 et 2 durent 0,1 seconde. P exécute à $t=10$ secondes un appel système qui dure 0,2 seconde puis à $t = 10,7$ une interruption a lieu dont le traitement dure 0,1 seconde. À quel moment les handlers des signaux sont appelés. Dessinez un chronogramme pour justifier votre réponse.

Solution:

Le handler du signal 2 sera appelé à $t=10,2s$ et le handler du signal 6 à $t=10,7+0,1= 10,8s$.

Question 3 1 point

Représenter en binaire la configuration du champs p_sig de P après l'exécution des deux handlers de la question précédente.

Solution:

15	...	7	6	5	4	3	2	1	0
0	...	0	1	0	0	0	0	0	0

Attention le bit du signal 7 reste.

Question 4 3 points

On veut réaliser l'appel système `int cancel_sig(int h)` qui supprime tous les signaux pendants du processus courant dont le handler est *ignoré* si $h = 0$ ou tous les signaux pendants dont le handler est positionné au traitement *par défaut* si $h \neq 0$. L'appel retourne le nombre de signaux supprimés.

Donnez le code de l'appel système `void sys_cancel_sig()` exécuté par le noyau lors de l'appel à `int cancel_sig(int h)`.

Solution:

```
void sys_cancel_sig() {
    struct proc *p=u.u_procp;
    int i,n,count=0;

    n = p->p_sig;
    for(i=1; i<NSIG; i++) {
        if (n&1)
            if(u.u_arg[0] == 0) {
```

```

        if (u.u_signal[i]&1) {
            p->p_sig &=~(1<<(i-1));
            count++;
        }
        else
            if (u.u_signal[i] == 0) {
                p->p_sig &=~(1<<(i-1));
                count++;
            }
        n >>= 1;
    }
    u.u_ar0[R0] = count;
}

```

Exercice 4 – Synchronisation (8 points)

On souhaite offrir l'appel système `int block_if_zomb_brothers()` qui bloque l'appelant à la priorité PZOMB tant qu'au moins un de ses frères est dans l'état zombi.

L'appel système renvoie l'erreur ENOPAR et ne bloque pas le processus appelant si le processus qui l'a créé (le père des frères) a déjà terminé. Sinon, il renvoie 0 si l'appelant n'a pas de frère ou aucun de ses frères se trouve dans l'état zombi. S'il y a au moins un processus zombi, l'appelant doit se bloquer.

Question 1 1½ points

Donnez le code d'une fonction interne `int zomb_brothers ( )` qui retourne 1 si le processus appelant a au moins un processus frère à l'état zombi. Sinon elle retourne 0.

Solution:

```

int zomb_brothers() {
    for (p = &proc[0]; p < &proc[NPROC]; p++)
        if (p->p_stat == SZOMB && p->p_ppid == u.u_procp->p_ppid)
            return 1;
    return 0;
}

```

Question 2 ½ point

Est-il possible que lorsqu'un processus bloqué dans `block_if_zomb_brothers` reprend son exécution, il a encore des frères zombis ? Justifiez votre réponse.

Solution:

Oui. Par exemple, un nouveau frère a été créé (fork) et il est devenu zombi avant que le processus débloquent s'exécute.

Question 3 3 points

Donnez le code de l'appel système `void sys_block_if_zomb_brothers()` exécuté par le noyau lors de l'appel à `int block_if_zomb_brothers()`.

Vous ne pouvez pas ajouter de champs supplémentaires dans la structure `proc`. En revanche, si nécessaire vous pouvez définir de nouveaux flags à mettre dans le champ `p_flag` de la structure `proc`.

Solution:

```
Ajout flag

#define PWAITBROT 32

void sys_wait_brother () {
struct proc *p = u.u_procp;

loop :
    if (p->p_ppid == 1) { /* Pas de père */
        u.u_error = ENOPAR;
        return;
    }
    if (zomb_brothers()){
        p->p_flag |= PWAITBROT;
        sleep ((caddr_t)p + 3 , PZOMB);
        p->p_flag&=~PWAITBROT;
        goto loop;
    }
    u.u_ar0[R0]=0;
}
```

Question 4 3 points

Modifiez le code la fonction la fonction wait donné en annexe pour gérer le réveil des processus bloqués dans `void sys_block_if_zomb_brothers()` lorsque ceux-ci n'ont de plus frère zombi. Vous indiquerez uniquement les lignes ajoutées ou modifiées.

Solution:

```
wait ()
{
    register f, *bp;
    register struct proc *p,*q;
    int nb_zomb = 0;
    f=0;
loop:
    for (p = &proc[0]; p < &proc [NPROC]; p++)
        if (p->p_ppid == u.u_procp -> p_pid) {
            f++;

            if (p->p_stat == SZOMB) {
                u.u_ar0[R0] = p->p_pid;
                bp = bread (swapdev, f =p->p_addr);
                mfree (swapmap, 1, f);
                p->p_stat = NULL;
                p->p_id = 0;
                p->p_ppid = 0;
                p->p_sig = 0;
                p_ttyp = 0;
                p->p_flag = 0;
                p = bp->b_addr;
                u.u_ar0[R1]= p->u_arg[0];
                brelse (bp);

                /* Ajout */
                for (p = &proc[0]; p < &proc[NPROC] ; p++)
```

```

        if (p->p_stat==SZOMB && p->p_ppid == u.u_procp->p_pid)
            nb_zomb++;

        if (!nb_zomb)
            /* plus de zombi, reveiller les fils en attente */
            for (p=&proc[0]; p<&proc[NPROC] ; p++)
                if (p->p_stat != NULL && p->p_ppid == u.u_procp->p_pid && p->p_flag
                    &PWAITBROT)
                    wakeup( (caddr_t) p+3 );

        /* Fin ajout*/

        return;
    }
}

if (f)
{
    sleep (u.u_procp, PWAIT);
    goto loop;
}
u.u_error = ECHILD;
}

```

```

struct  proc
{
    short  p_addr;      /* address of swappable image */
    short  p_size;      /* size of swappable image (in blocks) */
    int     p_flag;      /* process flags */
    char    p_stat;      /* process state */
    char    p_pri;       /* priority, negative is high */
    char    p_nice;      /* nice for scheduling */
    short   p_sig;       /* signals pending to this process */
    short   p_uid;       /* real user id, used to direct tty signals */
    short   p_suid;      /* set (effective) user id */
    short   p_time;      /* resident time for scheduling */
    int     p_cpu;       /* cpu usage for scheduling */
    short   *p_ttyp;     /* controlling tty */
    short   p_pid;       /* unique process id */
    short   p_ppid;      /* process id of parent */
    caddr_t p_wchan;     /* event process is awaiting */
    struct  text *p_textp; /* pointer to text structure */
    short   p_tsize;     /* size of text */
    short   p_ssize;     /* size of stack */
    short   p_clktim;    /* time to alarm clock signal */
} proc[NPROC];

/* stat codes */
#define SSLEEP 1 /* awaiting an event */
#define SRUN 3 /* running */
#define SZOMB 5 /* intermediate state in process termination */

struct  user
{
    int     u_rsav[2];    /* saved env. for process switching */
    int     u_ssav[2];    /* saved env. for swapping */
    int     u_qsav[2];    /* saved env. for signaling */
    struct  proc *u_procp; /* pointer to proc structure */
    char    u_error;      /* return error code */
    char    u_intflg;     /* catch intr from sys */
    int     *u_ar0;       /* address of users saved R0 */
    int     u_arg[20];    /* arguments to current system call */
    int     *u_ap;        /* pointer to arglist */
    struct  file *u_ofile[NOFILE]; /* pointers to open file */
    int     u_signal[NSIG]; /* disposition of signals */
    int     u_uid;        /* effective user id */
    int     u_gid;        /* effective group id */
    int     u_ruid;       /* real user id */
    int     u_rgid;       /* real group id */
    int     u_tsize;      /* text size (in blocs) */
    int     u_dsize;      /* data size (in blocs) */
    int     u_ssize;      /* stack size (in blocs) */
    int     u_csize;      /* amount of stack in use (in blocs) */

    long    u_utime;      /* this process user time */
    long    u_stime;      /* this process system time */
    long    u_cutime;     /* sum of childs' utimes */
    long    u_cstime;     /* sum of childs' stimes */
    ...
} u;

issig()
{

```

```

    register n;
    register struct proc *p;

    p = u.u_procp;
    while(p->p_sig) {
        n = fsig(p);
        if((u.u_signal[n]&1) == 0)
            return(n);
        p->p_sig &= ~(1<<(n-1));
    }
    return(0);
}

/*
 * Perform the action specified by
 * the current signal.
 * The usual sequence is:
 * if(issig())
 *   psig();
 */
psig()
{
    register n, p;
    register struct proc *rp;

    rp = u.u_procp;
    n = fsig(rp);
    if (n==0)
        return;
    rp->p_sig &= ~(1<<(n-1));
    if((p=u.u_signal[n]) != 0) {
        u.u_error = 0;
        u.u_signal[n] = 0;
        sendsig(p, n);
        return;
    }
    switch(n) {

        case SIGQUIT:
        case SIGINS:
        case SIGTRC:
        case SIGIOT:
        case SIGEMT:
        case SIGFPT:
        case SIGBUS:
        case SIGSEG:
        case SIGSYS:
            if(core())
                n += 0200;
    }
    exit(n);
}

/*
 * find the signal in bit-position
 * representation in p_sig.
 */
fsig(p)
struct proc *p;
{

```



```

        register n, i;

        n = p->p_sig;
        for(i=1; i<NSIG; i++) {
            if(n & 1)
                return(i);
            n >>= 1;
        }
        return(0);
    }
}

1
2 wait ()
3 {
4     register f, *bp;
5     register struct proc *p;
6
7     f=0;
8 loop:
9     for (p = &proc[0]; p < &proc [NPROC]; p++)
10        if (p->p_ppid == u.u_procp -> p_pid) {
11            f++;
12
13            if (p->p_stat == SZOMB) {
14                u.u_ar0[R0] = p->p_pid;
15                bp = bread (swapdev, f =p->p_addr);
16                mfree (swapmap, 1, f);
17                p->p_stat = NULL;
18                p->p_id = 0;
19                p->p_ppid = 0;
20                p->p_sig = 0;
21                p_ttyp = 0;
22                p->p_flag = 0;
23                p = bp->b_addr;
24                u.u_ar0[R1]= p->u_arg[0];
25                brelse (bp);
26                return;
27            }
28        }
29
30    if (f)
31    {
32        sleep (u.u_procp, PWAIT);
33        goto loop;
34    }
35    u.u_error = ECHILD;
36 }

```